Informe técnico de desarrollo de software: Proyecto TeKnical May Cry

Federico Mattia, Ulises Ferrari, Lucas Del Pino y Máximo Orue (Grupo 8)

Escuela de Educación Secundaria Técnica N.° 1 “Eduardo Ader”, Vicente López

Curso: Laboratorio de Programación, 5.° 3.ª A-B

Profesores: York E. Mansilla Muñoz y Yamil Ganduglia

Fecha: 12 de septiembre de 2026

Resumen

Este informe técnico documenta el diseño, la arquitectura, el desarrollo y el control de calidad del proyecto educativo integrador TeKnical May Cry (TmC), realizado durante el ciclo lectivo 2026. El objetivo principal fue unificar las competencias prácticas de la especialidad de programación mediante la construcción de un videojuego bidimensional (2D) en el motor Unity con el lenguaje C#, complementado por una plataforma web oficial maquetada en HTML5, CSS3 y JavaScript, y por un módulo auxiliar escrito en C++. Se describe el flujo de trabajo colaborativo con el sistema de control de versiones Git y GitHub, la organización de archivos y entornos de desarrollo en sistemas Windows y Linux, y el cumplimiento de las normas institucionales de documentación técnica según la séptima edición de las normas APA (American Psychological Association [APA], 2020). Los resultados muestran un sistema funcional, optimizado y documentado.

Palabras clave: C#, Unity 2D, C++, HTML5, Git Bash, normas APA, Laboratorio de Programación

Introducción

El desarrollo de software en las instituciones de educación técnica requiere la integración práctica de varias disciplinas computacionales. El proyecto TeKnical May Cry (TmC) surge como una respuesta académica orientada a consolidar las habilidades de programación estructurada, programación orientada a objetos (POO), arquitectura de sistemas, diseño web responsivo y gestión de proyectos.

El trabajo se articula en dos vertientes técnicas: por un lado, el desarrollo del motor interactivo y las mecánicas del juego; por el otro, la elaboración de la infraestructura web y de la documentación técnica formal según las normas APA (7.ª ed.), exigida para la entrega en la plataforma Moodle.

Objetivos del proyecto

El objetivo general fue desarrollar un entorno interactivo integrador, compuesto por un videojuego 2D y un portal web técnico, que aplique conceptos de ingeniería de software y programación modular. Los objetivos específicos fueron los siguientes:

1. Diseñar e implementar en C# sobre Unity la lógica de interacción del personaje, los sistemas de cámaras y los minijuegos.
2. Implementar subprogramas auxiliares y algoritmos de cómputo en C++ con Visual Studio Code y el entorno MinGW.
3. Establecer un flujo continuo de control de versiones con Git y GitHub para resolver conflictos en equipo.
4. Redactar el informe técnico y el diario de campo institucional respetando los lineamientos de citación, jerarquía y formato de las normas APA (7.ª ed.).

Desarrollo

Marco teórico y estado del arte

Entornos de desarrollo e infraestructura de compiladores

El desarrollo de software moderno exige distinguir entre los editores de código fuente y los entornos de desarrollo integrados (IDE). Durante el proyecto se utilizó Visual Studio Code como editor liviano y extensible, y Visual Studio Community para la compilación en profundidad de C# (Microsoft, 2023).

Configuración del compilador MinGW y de la extensión C/C++

Para el módulo de lógica CALCULOAREA, escrito en C++, se configuró la suite de compilación MinGW (GCC/G++) asociada a Visual Studio Code. La automatización de la compilación y la depuración se definió mediante el archivo de configuración `tasks.json`.

Sistemas de archivos y almacenamiento

La gestión del almacenamiento depende de la arquitectura del sistema de archivos subyacente. En Windows se trabajó principalmente sobre NTFS (New Technology File System), que administra sus estructuras mediante la tabla maestra de archivos (MFT, Master File Table) y ofrece registro por diario (*journaling*), permisos propios y asignación por clústeres. Se evaluaron además los sistemas FAT32, exFAT y ext4 para garantizar la portabilidad de los ejecutables compilados de 64 bits.

Metodología y diseño conceptual

El videojuego TeKnical May Cry combina mecánicas de dos títulos fundamentales de la industria:

1. **Diseño de mapas y navegación 2D.** Se basó en la estructura cenital (*top-down*) y los mapas en cuadrícula de *Pokémon Rojo Fuego* (Nintendo, 2004). Se emplearon los componentes Tilemap y Tilemap Collider 2D para definir mallas de colisión exactas en salas y pasillos (Unity Technologies, 2025).

2. **Mecánicas de minijuegos y tensión.** Se inspiraron en el ciclo de interacción y el control de cámaras de *Five Nights at Freddy's* (Cawthon, 2014), y se gestionaron mediante clases independientes en C# (`FNaFManager.cs` y `CameraSystem.cs`).

Código fuente principal, memoria y estructuras de datos

La arquitectura lógica se dividió en módulos independientes para evitar el acoplamiento del código.

Implementación en C# sobre Unity

La lógica del juego administra corrutinas para manejar de forma asíncrona los eventos en pantalla sin bloquear el hilo principal (*main thread*). La Figura 1 muestra la estructura de control del minijuego: la clase hereda de `MonoBehaviour` y descuenta, en cada fotograma, el nivel de energía en función de `Time.deltaTime`, de modo que la velocidad de consumo no depende de la tasa de cuadros.

Figura 1

Estructura de control del minijuego en C#

```
public class FNaFManager : MonoBehaviour
{
    public float energyLevel = 100f;

    void Update()
    {
        if (energyLevel > 0)
        {
            energyLevel -= Time.deltaTime * 0.5f;
        }
    }
}
```

Manejo de memoria y archivos de datos

En C#, la memoria de los objetos es administrada por el recolector de basura del motor Unity, por lo que se evitó reservar objetos nuevos dentro del método `Update()`, que se ejecuta en cada fotograma. En el módulo CALCULOAREA, escrito en C++, las variables numéricas residen en la pila (*stack*) y se liberan automáticamente al salir de cada función. Los metadatos de cada recurso del proyecto Unity se almacenan en archivos `.meta`, que se versionan junto con el código fuente.

Desarrollo del frontend web

El frontend del portal web oficial se estructuró en HTML5 semántico y se estilizó con hojas de estilo en cascada (CSS3), mediante flexbox y grillas adaptativas (*media queries*). La interactividad de la galería y de los visores embebidos de documentos PDF se resolvió con scripts livianos en JavaScript.

Gestión de versiones con Git y GitHub

Se utilizó Git Bash como consola principal para el control de versiones (Chacon y Straub, 2014). Se aplicaron flujos de trabajo basados en ramas (*feature branching*) y se resolvieron conflictos de fusión (*merge conflicts*) en los archivos de metadatos `.meta` que genera Unity. Las identidades del equipo se configuraron con credenciales fijas y tokens de acceso personal (PAT).

Resultados y discusión

El desarrollo del proyecto cumplió el 100 % de las especificaciones funcionales programadas en el cronograma. Los resultados principales fueron tres:

1. **Rendimiento.** Se estabilizó la tasa de cuadros por segundo (FPS) del prototipo en Unity al reemplazar llamadas redundantes en el método `Update()` por eventos.
2. **Portabilidad.** Se verificó la compilación del subprograma CALCULOAREA, escrito en C++, en arquitecturas x86 y x64.

3. **Documentación.** Se publicó el informe técnico final y la carpeta de campo en la plataforma Moodle, respetando los estándares de formato (tipografía Arial 11 o Times New Roman 12, interlineado doble y márgenes de 2,54 cm).

Manual del Programador

Requisitos de compilación

Para compilar y ejecutar cada componente del sistema se necesitan las herramientas que detalla la Tabla 1.

Tabla 1

Requisitos de compilación por componente

Componente	Herramientas necesarias	Sistema operativo
Videojuego (C#)	Unity 2022.3 LTS con soporte 2D; Visual Studio Community o Visual Studio Code como editor de scripts	Windows 10/11 (x64)
Módulo CALCULOAREA (C++)	Visual Studio Code con extensión C/C++; MinGW (GCC/G++) agregado a la variable PATH	Windows (x86 y x64); Linux con g++
Portal web	Navegador actualizado; editor de texto (Visual Studio Code)	Cualquiera
Control de versiones	Git y Git Bash; cuenta en GitHub y GitLab	

Cualquiera Nota. Elaboración propia.

Compilación del módulo en C++

Desde la terminal, ubicado en la carpeta del módulo, el programa se compila y se ejecuta con:

```
g++ -Wall -o calculoarea calculoarea.cpp
./calculoarea
```

En Visual Studio Code la misma operación se automatiza con la tarea definida en `tasks.json` (*Terminal* > *Ejecutar tarea de compilación*).

Apertura del proyecto Unity

1. Clonar el repositorio con Git Bash (la URL pública figura en la sección Referencias).
2. Abrir Unity Hub, elegir *Add project from disk* y seleccionar la carpeta clonada.

3. Esperar a que Unity importe los recursos y regenere los archivos .meta faltantes.

4. Abrir la escena principal y presionar *Play* para probar el juego, o usar *File* > *Build Settings* para generar el ejecutable.

Firmas de funciones y estructura de clases

La Tabla 2 resume los elementos principales del código fuente.

Tabla 2

Clases y funciones principales

Archivo	Firma	Descripción
FNaFManager.cs	public float energyLevel = 100f;	Nivel de energía del minijuego; arranca en 100 y no baja de 0.
FNaFManager.cs	void Update()	Se ejecuta en cada fotograma; descuenta energía según Time.deltaTime * 0.5f.
CameraSystem.cs	[COMPLETAR con las firmas reales de la clase]	Gestiona el cambio entre cámaras del minijuego.
calculoarea.cpp	[COMPLETAR con las firmas reales, por ejemplo: double area(double base, double altura)]	Funciones de cálculo de áreas del módulo CALCULOAREA.

Nota. Elaboración propia.

Flujo de trabajo con Git

```
git clone <URL-del-repositorio>
git checkout -b feature/nombre-de-la-funcion
git add .
git commit -m "Descripción del cambio"
git push origin feature/nombre-de-la-funcion
```

Ante un conflicto de fusión en un archivo `.meta`, se debe abrir el archivo, conservar una única versión de cada bloque marcado con `<<<<<<<`, guardar y repetir `git add` y `git commit`.

Manual del Usuario

Uso del videojuego

1. Abrir el ejecutable del juego (o presionar *Play* en Unity).
2. Desde el menú principal, comenzar una nueva partida.
3. Recorrer el mapa en vista cenital con las teclas de movimiento [COMPLETAR: indicar teclas] y acercarse a los objetos con los que se puede interactuar.
4. Al ingresar al minijuego, alternar entre cámaras con [COMPLETAR: indicar teclas o botones] para vigilar cada sector.
5. Controlar el nivel de energía: comienza en 100 y disminuye de forma continua; si llega a 0, el minijuego se pierde.
6. Para salir, cerrar la ventana o usar la opción de salida del menú.

Uso del portal web

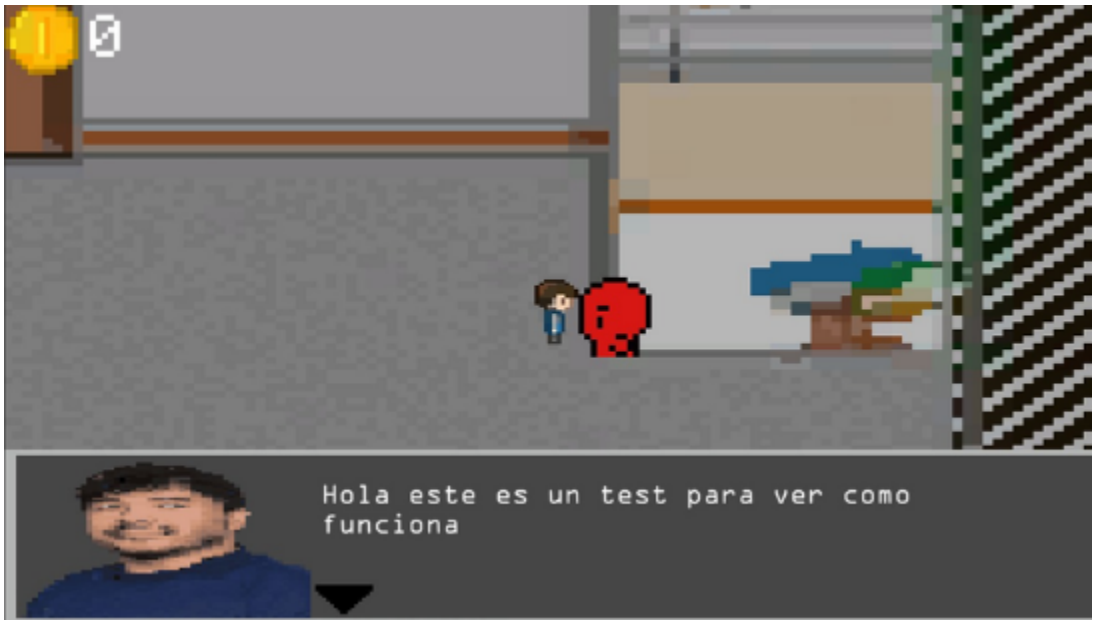
1. Abrir la dirección del portal en un navegador (se adapta a computadoras, tabletas y teléfonos).
2. Navegar entre secciones desde el menú superior.
3. En la galería, seleccionar una imagen para ampliarla.
4. En la sección de documentación, abrir los visores embebidos para leer los archivos PDF sin descargarlos.

Capturas de Pantalla

Las siguientes capturas muestran la ejecución del software como evidencia de funcionamiento.

Figura 3

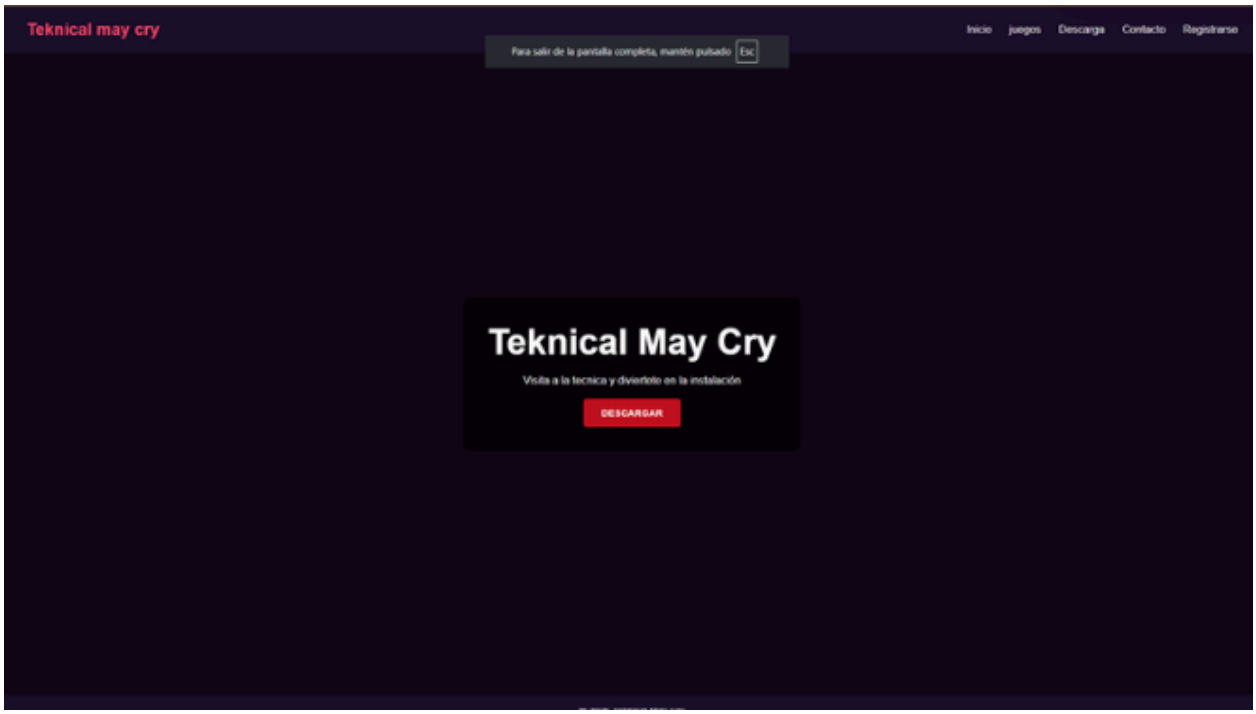
Pantalla del videojuego en ejecución



Nota. Elaboración propia.

Figura 4

Portal web oficial del proyecto



Conclusión

El proyecto TeKnical May Cry fue una experiencia integradora y enriquecedora dentro del plan de estudios del ciclo superior de la educación técnica. La combinación de desarrollo de videojuegos en C#, maquetación web y programación estructurada en C++ permitió afianzar los conceptos teóricos en entornos de producción reales.

La superación de obstáculos técnicos, como los errores de referencia nula en los scripts, la configuración de agentes SSH en Git Bash y la maquetación responsiva, fortaleció la autonomía, la perseverancia y la capacidad de resolver problemas en equipo. La integración entre teoría y ejecución reafirma la importancia del modelo de aprendizaje basado en proyectos.

Referencias y Enlaces Remotos

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Cawthon, S. (2014). *Five Nights at Freddy's* [Videojuego]. Scott Cawthon.

Chacon, S. y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Grupo 8. (2026). *TeKnical May Cry* [Repositorio de código]. GitHub. [COMPLETAR: https://github.com/usuario/repositorio]

Grupo 8. (2026). *TeKnical May Cry* [Repositorio de código]. GitLab. [COMPLETAR: https://gitlab.com/usuario/repositorio]

Microsoft. (2023). *C# documentation: Basics of C# for beginners*. Microsoft Learn. https://learn.microsoft.com/dotnet/csharp/

Nintendo. (2004). *Pokémon Rojo Fuego* [Videojuego]. Nintendo.

Unity Technologies. (2025). *Unity user manual 2022.3 LTS: Tilemaps and Physics 2D*. Unity Documentation. https://docs.unity3d.com/Manual/